Chapter 4
Understanding Loss Functions and Training Dynamics

Objective: To understand the role of loss functions in training a neural network, how to interpret loss values during training, and the process of optimization. This session will also involve analyzing and tweaking the training process to improve model performance.

1. What Are Loss Functions?

A loss function measures how well the model's predictions match the true labels. During training, the optimizer updates the model's weights to minimize this loss.

Tasks:

1. Research the difference between **Cross-Entropy Loss** and **Mean Squared Error (MSE)**. Why is Cross-Entropy Loss more suitable for classification tasks like CIFAR-10?
2. Look at the formula for Cross-Entropy Loss. What does it penalize? How does it handle incorrect predictions with high confidence?

Questions to Explore:

- What does a lower loss value mean for the model's performance?
- How is the loss function connected to the gradients used in backpropagation?

3. Training Dynamics

Tasks:

1. Look at the training and validation loss/accuracy plots from Chapter 3. Answer the following:
    - Are the training and validation loss decreasing over epochs?
    - Is the gap between training and validation accuracy increasing? What might this indicate?
2. Research the terms **overfitting** and **underfitting**. Based on your plots, is your model showing signs of either? Why?

Questions to Explore:

- What might cause the validation accuracy to plateau while training accuracy continues to improve?
- How could you adjust the model or training process to address overfitting or underfitting?

3. Experimenting with Hyperparameters

Hyperparameters like learning rate, batch size, and number of epochs significantly influence the training process.

Tasks:

1. Experiment with different learning rates (e.g., 0.1, 0.01, 0.001). How does a larger or smaller learning rate affect convergence and final accuracy?
2. Vary the batch size (e.g., 32, 64, 128). How does changing the batch size influence training time and performance?
3. Try training the model for fewer or more epochs. At what point does the validation loss start to increase, indicating overfitting?

Programming Exercise:
Modify the **model.fit()** call from Chapter 3 to experiment with these hyperparameters. Record your observations for each run.

4. Regularization Techniques

To improve generalization and prevent overfitting, you can use techniques like dropout and weight regularization.

Tasks:

1. Research **dropout layers**. What do they do during training? How can they help prevent overfitting?
2. Research **L2 regularization** (also called weight decay). What effect does it have on the model's weights?

Programming Exercise:

- Add a dropout layer to your model (e.g., Dropout(0.5) after a hidden layer).
- Add L2 regularization to one of the dense layers (e.g., Dense(units, kernel_regularizer=tf.keras.regularizers.l2(0.01))).
- Retrain the model and compare the training and validation accuracy before and after adding these regularization techniques.

5. Reflection on Training Dynamics

After running experiments and analyzing your results, reflect on the following:
Questions to Reflect On:

1. How did changing the learning rate affect the model's ability to converge?
2. What effect did dropout and L2 regularization have on the model's performance?
3. How do you decide the best combination of hyperparameters for your model?

Programming Assignment:

Write a Python script to:

1. Experiment with at least two different learning rates.
2. Add a dropout layer to your FCNN model.
3. Train the modified model and compare its performance to the original model.

Below is a **skeleton code snippet** for the dropout modification:

```
from tensorflow.keras.layers import Dropout
from tensorflow.keras.regularizers import l2

# Modify the model with dropout and L2 regularization
model = Sequential([
  Flatten(input_shape=(32, 32, 3)),
  Dense(128, activation='relu', kernel_regularizer=l2(0.01)),  # L2 regularization
  Dropout(0.5),  # Dropout layer
  Dense(64, activation='relu'),
  Dense(10, activation='softmax')
])

# Compile and train the model
model.compile(
  optimizer=Adam(learning_rate=0.001),
  loss='sparse_categorical_crossentropy',
  metrics=['accuracy']
)

history = model.fit(
  train_images, train_labels,
  validation_data=(val_images, val_labels),
  epochs=20,
  batch_size=64
)
```